Analisador de Tráfego — Como baixar e executar

Oct 22, 2026 · @Rafael de Souza Rodrigues

Antes de começar

O Analisador de Tráfego roda em Docker, então o único programa obrigatório é o Docker. Tudo o mais (Python, Scapy, libpcap) já vem dentro da imagem.

Sistema	O que instalar	O que a captura enxerga
Linux (Ubuntu, Debian, Fedora etc.)	Docker Engine com o plugin Compose	O tráfego real da placa de rede da máquina
Windows 10 22H2 ou Windows 11	WSL2 e Docker Desktop	O tráfego da máquina virtual do Docker, não o do navegador

Opcional nos dois sistemas: o **Git**, para baixar o projeto pela linha de comando, e um visualizador de SQLite, para ver os pacotes gravados.

Repositório: `https://github.com/SEU-USUARIO/Analise_de_Trafego`

Nos comandos deste guia, troque `SEU-USUARIO` pelo usuário do GitHub onde o projeto está publicado.

Baixar o projeto do GitHub

Há duas formas de baixar, e as duas dão o mesmo resultado. Use o Git se ele já estiver instalado; caso contrário, baixe o ZIP.

Opção 1: com Git

No terminal (Linux) ou no PowerShell (Windows):

```
git clone https://github.com/SEU-USUARIO/Analise_de_Trafego.git
cd Analise_de_Trafego
```

Opção 2: pelo ZIP, sem Git

1. Abra o endereço do repositório no navegador.
2. Clique no botão verde **Code** e depois em **Download ZIP**.

3. Extraia o arquivo. No Windows, use uma pasta sem espaços nem acentos, por exemplo `C:\projetos`.

O GitHub coloca `-main` no nome da pasta do ZIP, então ela se chamará `Analise_de_Trafego-main`. Isso não atrapalha a aplicação.

Confira que está na pasta certa

Todos os comandos deste guia são executados na pasta que contém o arquivo `docker-compose.yml`. Ao listar o conteúdo dela, você deve ver:

```
app/  data/  tests/  Dockerfile  docker-compose.yml  README.md
requirements.txt
```

Se aparecer só outra pasta com o mesmo nome, entre nela. Isso acontece quando o ZIP é extraído dentro de uma pasta já criada.

Linux

No Linux, o container usa a rede da própria máquina, então a captura mostra o tráfego real da placa de rede. Os comandos abaixo foram pensados para Ubuntu e Debian; em outras distribuições, só a instalação do Docker muda.

1. Instalar o Docker

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
```

O segundo comando permite usar o Docker sem `sudo`. Para ele valer, **saia da sessão e entre de novo** (ou reinicie). Depois confira:

```
docker version
docker compose version
```

2. Construir a imagem e rodar os testes

Na pasta do projeto:

```
docker compose build
docker compose run --rm --entrypoint pytest analyzer -v
```

O resultado esperado dos testes é `9 passed`.

3. Descobrir o nome da interface

```
docker compose run --rm analyzer interfaces
```

No Linux, os nomes variam: `eth0`, `enp3s0` (cabo) ou `wlp2s0` (Wi-Fi). Use o nome marcado como `(padrão)`.

4. Capturar

Abra dois terminais na pasta do projeto. No **terminal 1**, inicie a captura de 60 segundos, trocando `eth0` pelo nome da sua interface:

```
docker compose run --rm analyzer capture -i eth0 -t 60
```

No **terminal 2**, enquanto a captura roda, gere tráfego:

```
ping -c 20 8.8.8.8
curl -s https://example.com > /dev/null
```

Navegar em sites durante a captura também funciona, porque no Linux o tráfego do navegador passa pela mesma placa.

5. Consultar os resultados

```
docker compose run --rm analyzer sessions   # lista as capturas
docker compose run --rm analyzer stats      # relatório da última captura
```

Para ver os pacotes gravados, um por um, use o `sqlite3`. O `sudo` é necessário porque o arquivo foi criado pelo container como root:

```
sudo apt install sqlite3
sudo sqlite3 -header -column data/traffic.db "SELECT * FROM packets ORDER BY
id DESC LIMIT 20;"
```

Windows

No Windows, o Docker roda dentro de uma máquina virtual Linux. A captura mostra o tráfego dessa máquina virtual, então o tráfego de teste precisa ser gerado por um container, e não pelo navegador.

1. Verificar a virtualização

Abra o Gerenciador de Tarefas (Ctrl+Shift+Esc), vá em **Desempenho** e depois em **CPU**. Deve aparecer "Virtualização: Habilitado". Se estiver desabilitado, ative a opção Intel VT-x ou AMD-V na BIOS do computador.

2. Instalar o WSL2

Abra o PowerShell **como Administrador** (clique com o botão direito no menu Iniciar) e rode:

```
wsl --install
```

Reinicie o computador quando terminar.

3. Instalar o Docker Desktop

1. Baixe em docker.com/products/docker-desktop e instale, deixando marcada a opção **Use WSL 2**.
2. Abra o Docker Desktop e espere o ícone da baleia, perto do relógio, parar de se mexer.
3. Deixe o Docker Desktop aberto durante todo o uso da aplicação.

4. Abrir o PowerShell na pasta do projeto

1. No Explorador de Arquivos, entre na pasta do projeto, a que tem o `docker-compose.yml`.
2. Clique na barra de endereço, apague o texto, digite `powershell` e aperte Enter.

A janela que abrir deve terminar com o caminho da pasta, por exemplo `PS C:\projetos\Analise_de_Trafego>`.

5. Construir a imagem e rodar os testes

```
docker compose build
docker compose run --rm --entrypoint pytest analyzer -v
```

O resultado esperado dos testes é `9 passed`.

6. Capturar

No **PowerShell 1**, confira o nome da interface (normalmente `eth0`) e inicie a captura de 60 segundos:

```
docker compose run --rm analyzer interfaces
docker compose run --rm analyzer capture -i eth0 -t 60
```

Abra um **PowerShell 2** e, enquanto a captura roda, gere tráfego por um container:

```
docker run --rm --network host alpine sh -c "ping -c 20 8.8.8.8; wget -qO-
https://example.com > /dev/null"
```

7. Consultar os resultados

```
docker compose run --rm analyzer sessions
docker compose run --rm analyzer stats
```

Para ver os pacotes um por um, instale o DB Browser for SQLite:

```
winget install DBBrowserForSQLite.DBBrowserForSQLite
```

No programa, clique em **Open Database** e abra o arquivo `data\traffic.db` **dentro da pasta do projeto**. Na aba **Execute SQL**, rode:

```
SELECT * FROM packets ORDER BY id DESC LIMIT 50;
```

Se o banco estava aberto durante a captura, feche e abra de novo para ver os dados novos.

Opcional: capturar o tráfego real do Windows

Para ver o tráfego do navegador e da placa Wi-Fi, rode a aplicação sem Docker:

1. Instale o Python 3.12 (python.org), marcando **Add python.exe to PATH**.
2. Instale o Npcap (npcap.com) com as opções padrão.
3. Na pasta do projeto, rode:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
python -m app.main interfaces
python -m app.main capture -i "Wi-Fi" -t 60
```

Se o PowerShell bloquear o `Activate.ps1`, rode antes `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`. Se aparecer erro de permissão na captura, abra o PowerShell como Administrador.

Como saber se funcionou

A captura funcionou quando o terminal termina com a linha `Captura finalizada: N pacotes gravados`, com N maior que zero, seguida do relatório. Durante a captura, um painel ao vivo é atualizado a cada 5 segundos.

Exemplo de relatório, de uma captura real no Windows com Docker Desktop:

```
================================================================
 RESULTADO - sessão #4 (eth0)
================================================================
  Total de pacotes capturados: 1114
  Volume total:                307.9 KB

  Pacotes por protocolo
  Protocolo  Pacotes      %     Bytes
  ---------  -------  -----  --------
  TCP           1074  96.4%  304.0 KB
  ICMP            40   3.6%    3.8 KB

  Top 5 IPs de ORIGEM (ordenado por bytes)
  #  Endereço IP    Pacotes     Bytes  % bytes
  -  -------------  -------  --------  -------
  1  192.168.65.7       530  244.8 KB    79.5%
  2  192.168.65.1       519   49.6 KB    16.1%
  3  104.20.23.154       13    7.2 KB     2.3%
  4  192.168.65.3        32    4.4 KB     1.4%
  5  8.8.8.8             20    1.9 KB     0.6%
================================================================
```

Os 40 pacotes ICMP são o ping para o `8.8.8.8`, e o `104.20.23.154` é, muito provavelmente, o acesso ao example.com. No Windows, os IPs `192.168.65.x` são a rede interna do Docker Desktop. No Linux, aparecem os IPs da sua rede local e os dos sites acessados.

Os pacotes ficam salvos em `data/traffic.db`, dentro da pasta do projeto, e continuam lá depois que o container é encerrado.

Problemas comuns

A maioria dos problemas vem de três causas: o Docker não está rodando, o comando foi executado na pasta errada ou o tráfego foi gerado onde a captura não enxerga.

Sintoma	Sistema	Causa	Solução
`no configuration file provided: not found`	Ambos	O comando não foi executado na pasta do `docker-compose.yml`	Entre na pasta certa (veja "Confira que está na pasta certa")
Erro de conexão com o daemon do Docker	Windows	Docker Desktop fechado ou ainda iniciando	Abra o Docker Desktop e espere o ícone ficar parado
`permission denied` ao rodar `docker`	Linux	O usuário ainda não está no grupo `docker`	Saia da sessão e entre de novo, ou use `sudo`
`Interface 'eth0' não encontrada`	Ambos	A interface tem outro nome	Rode `interfaces` e use o nome que aparecer
Captura com 0 pacotes	Windows	Tráfego gerado pelo navegador, fora da máquina virtual do Docker	Gere tráfego com o container `alpine` (passo 6)
O banco aparece vazio no DB Browser	Windows	Foi aberto outro `traffic.db`, de outra pasta, ou o banco estava aberto antes da captura	Abra o `data\traffic.db` da pasta onde rodou o `docker compose`; feche e abra de novo
`unable to open database file` no `sqlite3`	Linux	O arquivo pertence ao root	Use `sudo sqlite3 ...`
`docker compose build` falha ao baixar pacotes	Ambos	Proxy ou VPN corporativa	Desconecte a VPN ou configure o proxy no Docker

Se o problema persistir, rode a captura com `-v` para ver logs detalhados, por exemplo `docker compose run --rm analyzer -v capture -i eth0 -t 30`.